ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №9

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Статичні та класові методи, декоратори класів»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Для Appointment реалізуйте перевірку вигаданого номера статичним методом, альтернативний класовий конструктор і декоратор реєстрації типу запису. Не використовуйте реальні медичні дані.

КОД РОЗВ'ЯЗКУ

```
from abc import ABC, abstractmethod

# Словник для реєстрації типів записів за допомогою декоратора класу
APPOINTMENT_TYPES = {}

def register_appointment_type(cls):
    """Декоратор класу: реєструє тип запису за його унікальним атрибутом kind."""
    if hasattr(cls, "kind"):
        APPOINTMENT_TYPES[cls.kind] = cls
    return cls

class AuditableMixin:
    """Mixin-клас, що додає функціонал логування (аудиту) подій для запису."""

    def audit_record(self):
        """Повертає рядок для запису в журнал подій медичного центру."""
        return f"[АУДИТ] Запис №{self.number} (час: {self.time}): змінено стан або створено подію."

class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def __str__(self):
        return f"Послуга «{self.name}» ({self.duration_min} хв)"

    def __repr__(self):
        return f"Service(name={self.name!r}, duration_min={self.duration_min!r})"

    def __eq__(self, other):
```

```
            if not isinstance(other, Service):
                return NotImplemented
            return self.name == other.name and self.duration_min ==
other.duration_min

    class Client:
        """Модель пацієнта (клієнта) медичного центру."""

        def __init__(self, full_name, phone):
            self.full_name = full_name
            self.phone = phone

        def __str__(self):
            return f"{self.full_name} (тел: {self.phone})"

        def __repr__(self):
            return f"Client(full_name={self.full_name!r},
phone={self.phone!r})"

    @register_appointment_type
    class Appointment(AuditableMixin, ABC):
        """Абстрактний базовий клас запису на прийом із підтримкою
допоміжних методів рівня класу."""

        kind = "base"
        valid_statuses = {"підтверджено", "завершено", "скасовано"}
        created_count = 0

        def __init__(self, number, time, service, client,
status="підтверджено"):
            self.number = number
            self.time = time
            self.service = service
            self.client = client

            self._status = "підтверджено"
            self.status = status

            Appointment.created_count += 1

        @property
        def status(self):
            """Повертає поточний статус із захищеного внутрішнього
поля."""
            return self._status

        @status.setter
        def status(self, value):
            """Дозволяє встановити лише допустимий статус."""
```

```
            if value not in self.valid_statuses:
                raise ValueError(f"Невідомий статус: {value}")
            self._status = value

        @staticmethod
        def is_valid_number(number):
            """Статичний метод: перевіряє, чи є номер додатним
цілим числом."""
            return isinstance(number, int) and number > 0

        @classmethod
        @abstractmethod
        def from_dict(cls, data, service, client):
            """Абстрактний класовий метод: кожен нащадок реалізує
власний альтернативний конструктор."""
            pass

        @abstractmethod
        def preparation_note(self):
            """Абстрактний метод підготовки до прийому."""
            pass

        def __str__(self):
            return (
                f"Запис №{self.number} | Час: {self.time} | "
                f"Пацієнт: {self.client.full_name} | Послуга:
{self.service.name} | Статус: {self.status}"
            )

        def __repr__(self):
            return (
                f"{self.__class__.__name__}(number={self.number!r},
time={self.time!r}, "
                f"service={self.service!r}, client={self.client!r},
status={self.status!r})"
            )

        def __eq__(self, other):
            if not isinstance(other, Appointment):
                return NotImplemented
            return self.number == other.number

    @register_appointment_type
    class Consultation(Appointment):
        """Похідний клас для консультаційних прийомів лікаря."""

        kind = "consultation"
```

```
        def __init__(self, number, time, service, client,
doctor_office, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.doctor_office = doctor_office

        @classmethod
        def from_dict(cls, data, service, client):
            """Альтернативний конструктор для Consultation з
урахуванням кабінету."""
            return cls(
                number=data["number"],
                time=data["time"],
                service=service,
                client=client,
                doctor_office=data.get("doctor_office", "101"),
                status=data.get("status", "підтверджено")
            )

        def preparation_note(self):
            return f"Консультація у каб. №{self.doctor_office}:
очікуйте під кабінетом, прийдіть за 10 хв."

    @register_appointment_type
    class DiagnosticVisit(Appointment):
        """Похідний клас для діагностичних процедур."""

        kind = "diagnostic"

        def __init__(self, number, time, service, client,
equipment_code, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.equipment_code = equipment_code

        @classmethod
        def from_dict(cls, data, service, client):
            """Альтернативний конструктор для DiagnosticVisit з
урахуванням коду обладнання."""
            return cls(
                number=data["number"],
                time=data["time"],
                service=service,
                client=client,
                equipment_code=data.get("equipment_code", "EQ-
0000"),
                status=data.get("status", "підтверджено")
            )

        def preparation_note(self):
```

```
            return f"Діагностика (обладнання код:
{self.equipment_code}): дотримуйтеся інструкцій лікаря."

     # Демонстрація роботи
     if __name__ == "__main__":
         print("=== 1. Перевірка роботи @staticmethod ===")
         test_number = 105
         print(f"Чи є число {test_number} валідним номером запису?
{Appointment.is_valid_number(test_number)}")
         print(f"Чи є число -5 валідним номером запису?
{Appointment.is_valid_number(-5)}")

         print("\n=== 2. Перевірка роботи @classmethod
(Альтернативний конструктор) ===")
         s1 = Service("Консультація терапевта", 30)
         c1 = Client("Олександр Т.", "+380501112233")

         # Вихідні дані у вигляді словника із зазначенням кабінету
для Consultation
         row_data = {
             "number": 10,
             "time": "2026-11-06 11:00",
             "doctor_office": "204",
             "status": "підтверджено"
         }

         # Створюємо об'єкт Consultation через класовий метод
         consultation_from_dict = Consultation.from_dict(row_data,
s1, c1)
         print("Створено через Consultation.from_dict():")
         print(consultation_from_dict)
         print(f"Кабінет: {consultation_from_dict.doctor_office}")
         print(f"Тип створеного об'єкта:
{type(consultation_from_dict).__name__}")

         print("\n=== 3. Перевірка роботи декоратора класу та
реєстру типів ===")
         print("Зареєстровані типи записів у системі
(APPOINTMENT_TYPES):")
         for kind, cls_ref in APPOINTMENT_TYPES.items():
             print(f" - Ключ (kind): '{kind}' -> Клас:
{cls_ref.__name__}")
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

```
PS C:\Users\user\AppData\Local\Programs\Microsoft VS Code> & C:\Users\user\AppData\Local\Programs\Python\Python315\python.exe c:/Users/user/Downloads/Lab09.py
=== 1. Перевірка роботи @staticmethod ===
Чи є число 105 валідним номером запису? True
Чи є число -5 валідним номером запису? False

=== 2. Перевірка роботи @classmethod (Альтернативний конструктор) ===
Створено через Consultation.from_dict():
Запис №10 | Час: 2026-11-06 11:00 | Пацієнт: Олександр Т. | Послуга: Консультація терапевта | Статус: підтверджено
Кабінет: 204
Тип створеного об'єкта: Consultation

=== 3. Перевірка роботи декоратора класу та реєстру типів ===
Зареєстровані типи записів у системі (APPOINTMENT_TYPES):
 - Ключ (kind): 'base' -> Клас: Appointment
 - Ключ (kind): 'consultation' -> Клас: Consultation
 - Ключ (kind): 'diagnostic' -> Клас: DiagnosticVisit
PS C:\Users\user\AppData\Local\Programs\Microsoft VS Code>
```